Select & Distinct Statements

→ SELECT * from students; → whole table.

→ SELECT branch from students;

⇓

Only the branch column.

→ SELECT DISTINCT branch from students;

⇓

This selects only specific data from the branch.

→ SELECT * from employees WHERE emp_id=4;

⇓

This will retrieve the complete data ~~(all column)~~ about the 4th employee

→ SELECT * from employees WHERE empid BETWEEN 1 and 3.

⇓

Retrieves the data btwn (: 1 & 3 inclusive)

where emp_id ≤ 3.

Operators

* = (Equal)
* <> (Not equal)
* BETWEEN
* LIKE
* IN.

Scanned with OKEN Scanner

ORDER By

* This used to sort the result-set in ascending or descending order.

```
SELECT * Score
FROM ASSESSMENT - SCORES
WHERE, Score >=60
ORDERBY score ASC;
```

* ORDER BY sorts through the ASCII value.
* By Default the order is ASC

Update & Delete

* Modifies the existing records in a table.

Syntax→

```
UPDATE table-name
SET column1 = value1, etc...
WHERE condition( );
```

Ex:- `UPDATE employees SET salary=150000 dept='Management' WHERE emp-id=4;`

Scanned with OKEN Scanner

Delete

* DELETE FROM employees WHERE dept='Logistics';

TOP & LIMIT

```
SELECT column_name
FROM table_name
WHERE condition
ORDER BY column_names
LIMIT number;
```

* SELECT * from employees LIMIT 5;
  ↓
  retreives the first 5 data entries.

* SELECT * from employees ORDER BY name LIMIT 5;
  ↓
  retreives the first 5 data entries alphabetically.

min & max

```
SELECT MAX/MIN(column_name)
FROM table_name
WHERE condition;
```

Scanned with OKEN Scanner

SELECT 2nd or nth highest data entry

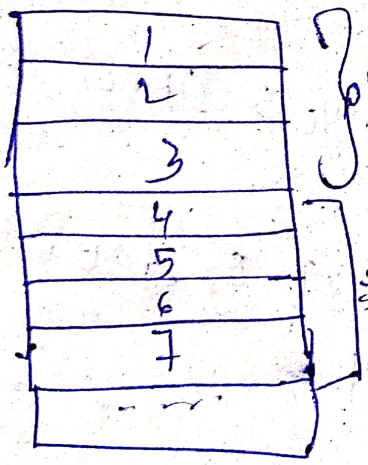
```
SELECT MAX(salary) FROM employee WHERE
salary < (SELECT MAX(salary) FROM
employee);
```

* This retreives the 2nd highest salary in the database.

Offset & Row count

→ 1, 2

offset — row count



Ex:- 3, 4

Syntax:-

```
SELECT column FROM table ORDER BY column
LIMIT offset, row_count;
```

Ex:-

```
SELECT salary FROM employees ORDER BY
salary LIMIT 3, 4;
```

Scanned with OKEN Scanner

COUNT(), AVG, SUM AVG/SUM.

SELECT COUNT(column-name)
FROM table-name
WHERE condition; } Syntax

* SELECT SUM(salary) FROM employees
  ⇓
  retrives sum of all people salaries.

* SELECT SUM(salary) FROM employees
  WHERE branch-id = 3;

Like & WildCards

* => Represents zero or more characters
? => Represents a single character
[] => Represents any single character within the brackets
! => Represents any character not in the brackets.
- => Represents a range of characters.
=> Represents any single numeric character

% => Represents zero or more characters

_ => Represents a single character

[] => Represents any single within brackets

^ => Represents any character not in the brackets.

– => Represents a range of characters.

Ex:-

SELECT * FROM employee WHERE name LIKE 'a%a'

Irrespective of how many letters that word consists it should start from a.

'%U%' => The word should contain "U" mandatorily.

'%ch%' => Should contain ch in the word.

'_i%' => Second character should be 'i'.

'_ch%' => Should be one character before ch

'mi_%' => First two characters should be mi & the next two characters can be anything from there many characters can take place

'm%t' => Starts with "m" & ends with "t".

* NOT LIKE '%a' ⇒ Name should not end with 'a'.

* '[dhm]%' ⇒ Should start with either d, h or m.

* '%[y]%' ⇒ ~~should be in middle~~ should contain "y" or "l".

* '%[hm][FS]%' ⇒ —— h (or) m, F (or) S ——

* '[a-d]%' ⇒ Any character b/w a & d is acceptable at the start

* '%[l-m]%' ⇒ b/w l-m any character is acceptable in the word.

NOT LIKE

*'[^am]%' ⇒ Name should not start with "a" or "m".

* '[!am]%' ⇒ Does not return the characters in the brackets.

→ Floor('b') ⇒ 0. (Floor of a String gives zero in any Sql).

→ Concat('sql','ql'); ⇒ This will return sql.ql

→ TO_NUMBER('5') ⇒ Converts a character value into a numeric datatype.

Scanned with OKEN Scanner